Unit-2

Unit-2

- **Searching**
- **Uninformed Search Strategies**
- **Informed (Heuristic) Search Strategies**
- **Game Playing- Adversial search**

Searching

2.1 Searching for solutions

- A solution is an action sequence, so search algorithms work by considering various possible action sequences. The possible action sequences starting at the initial state form a search tree with the initial state at the root; the branches are actions and the nodes correspond to states in the state space of the problem.
- The first few steps in growing the search tree for finding a route from Arad to Bucharest. The root node of the tree corresponds to the initial state, In(Arad).
    - ❑ The first step is to test whether this is a goal state.
    - ❑ Then we need to consider taking various actions. We do this by expanding the current state; that is, applying each legal action to the current state, thereby generating a new set of

Searching

- In this case, we add three branches from the parent node In(Arad) leading to three new child nodes: In(Sibiu), In(Timisoara), and In(Zerind). Now we must choose which of these three possibilities to consider further.

- Suppose we choose Sibiu first. We check to see whether it is a goal state (it is not) and then expand it to get In(Arad), In(Fagaras), In(Oradea), and In(RimnicuVilcea).
- We can then choose any of these four or go back and choose Timisoara or Zerind. Each of these six nodes is a leaf node, that is, a node with no children in the tree. The set of all leaf nodes available for expansion at any given point is called the frontier.

Searching

- The process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand.
- Search algorithms all share this basic structure; they vary primarily according to how they choose which state to expand next—the so-called search strategy.
- It includes the path from Arad to Sibiu and back to Arad again! We say that In(Arad) is a repeated state in the search tree, generated in this case by a loopy path. Considering such loopy paths means that the complete search tree for Romania is infinite because there is no limit to how often one can traverse a loop.
- On the other hand, the state space—the map shown in Figure 3.2—has only 20 states.

Searching

- Loops can cause certain algorithms to fail, making otherwise solvable problems unsolvable. Fortunately, there is no need to consider loopy paths. a loopy path to any given state is never better than the same path with the loop removed.
- Loopy paths are a special case of the more general concept of redundant paths, which exist whenever there is more than one way to get from one state to another. Consider the paths Arad–Sibiu (140 km long) and Arad–Zerind–Oradea–Sibiu (297 km long). Obviously, the second path is redundant—it's just a worse way to get to the same state

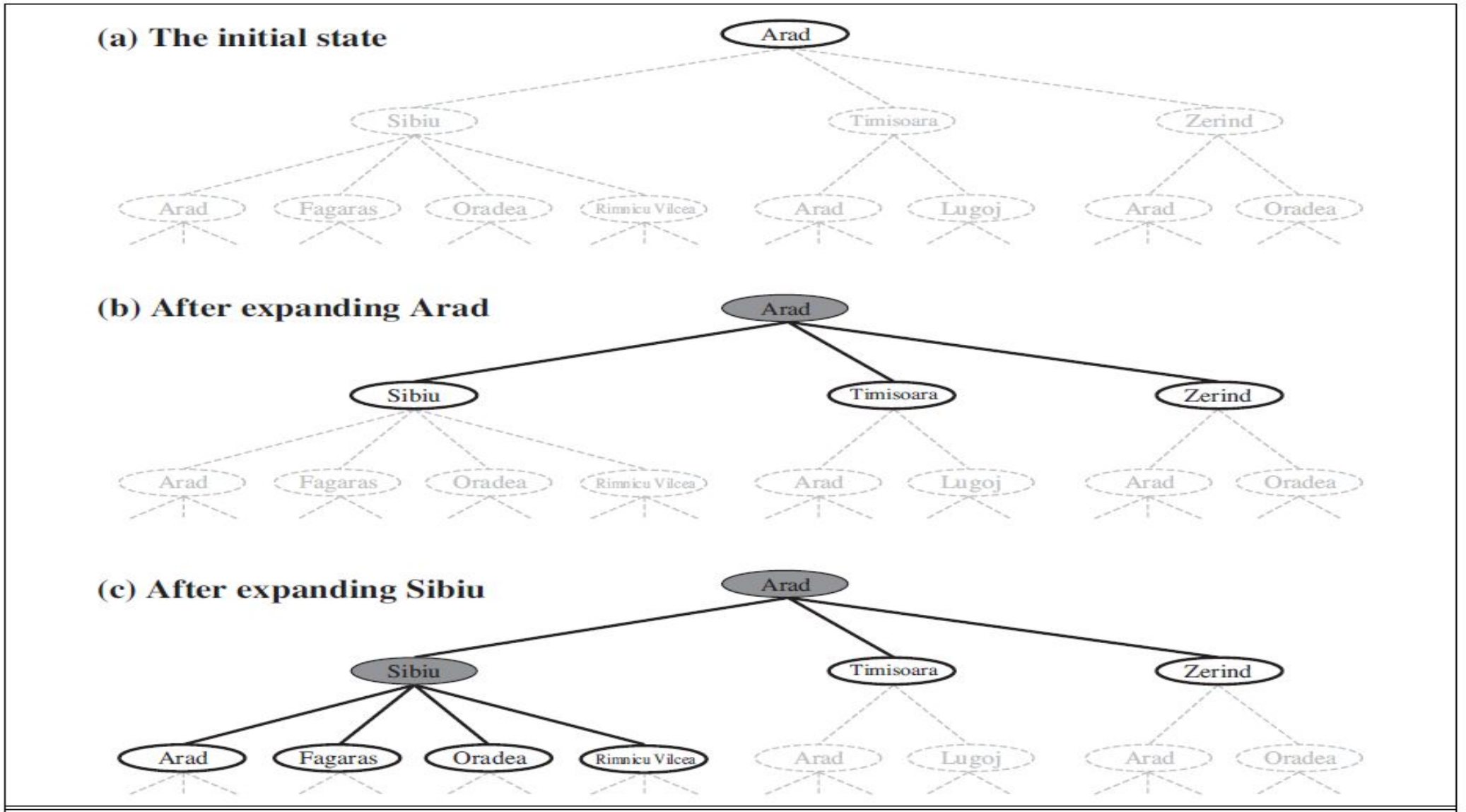


Figure 3.6 Partial search trees for finding a route from Arad to Bucharest. Nodes that have been expanded are shaded; nodes that have been generated but not yet expanded are outlined in bold; nodes that have not yet been generated are shown in faint dashed lines.

function TREE-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

---

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  initialize the explored set to be empty
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set

Figure 3.7 An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.

Searching

- In reversible problems like route-finding or sliding-block puzzles, redundant paths are unavoidable. Though there are only ~$2d^2$ distinct states within depth d, the search tree can have up to 4^d leaves—making tractable problems intractable due to redundancy, even with loop-avoidance.
- The way to avoid exploring redundant paths is to remember where one has been. To do this, we augment the TREE-SEARCH algorithm with a data structure called the explored set (also known as the closed list), which remembers every expanded node.
- Newly generated nodes that match previously generated nodes—ones in the explored set or the frontier—can be discarded instead of being added to the frontier

Searching

- Clearly, the search tree constructed by the GRAPH-SEARCH algorithm contains at most one copy of each state, so we can think of it as growing a tree directly on the state-space graph
- The frontier separates the state-space graph into the explored region and the unexplored region, ensuring all paths to unexplored states pass through it. Each step shifts a state from frontier to explored while adding new frontier states, enabling systematic exploration toward the solution.

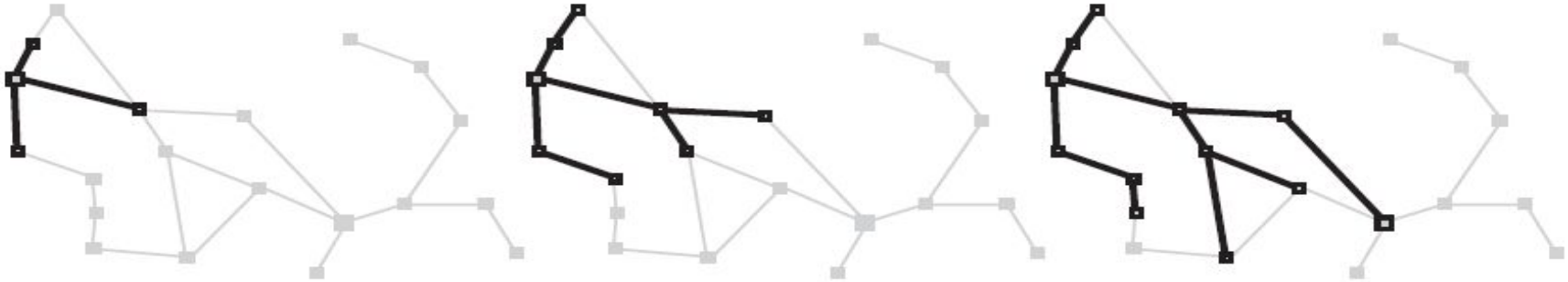


Figure 3.8 A sequence of search trees generated by a graph search on the Romania problem of Figure 3.2. At each stage, we have extended each path by one step. Notice that at the third stage, the northernmost city (Oradea) has become a dead end: both of its successors are already explored via other paths.

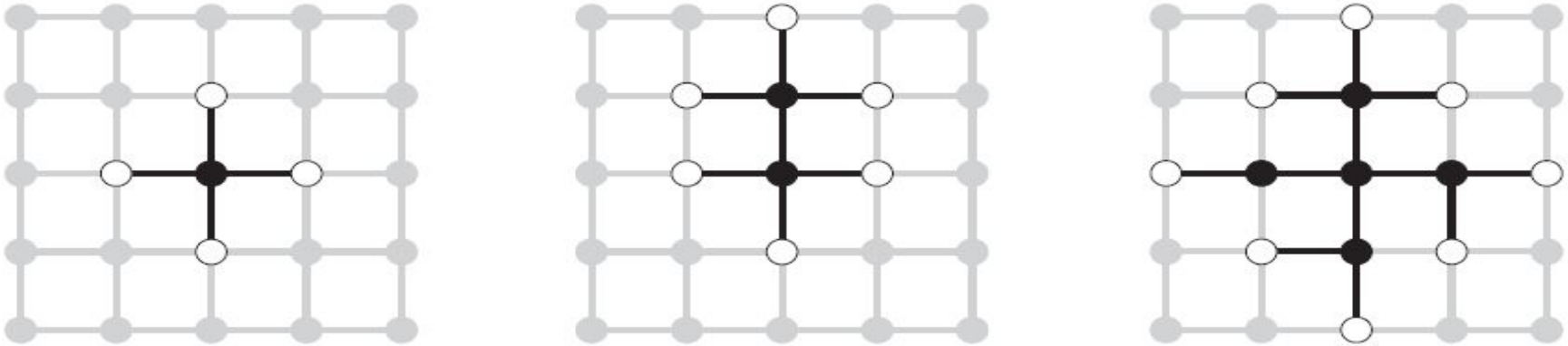


Figure 3.9 The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

2.1.1 Infrastructure for search algorithms

- Search algorithms require a data structure to keep track of the search tree that is being constructed.

- For each node n of the tree, we have a structure that contains four components:

  - n.STATE: the state in the state space to which the node corresponds;
  - n.PARENT: the node in the search tree that generated this node;
  - n.ACTION: the action that was applied to the parent to generate the node;
  - n.PATH-COST: the cost, traditionally denoted by g(n), of the path from the initial state to the node, as indicated by the parent pointers.

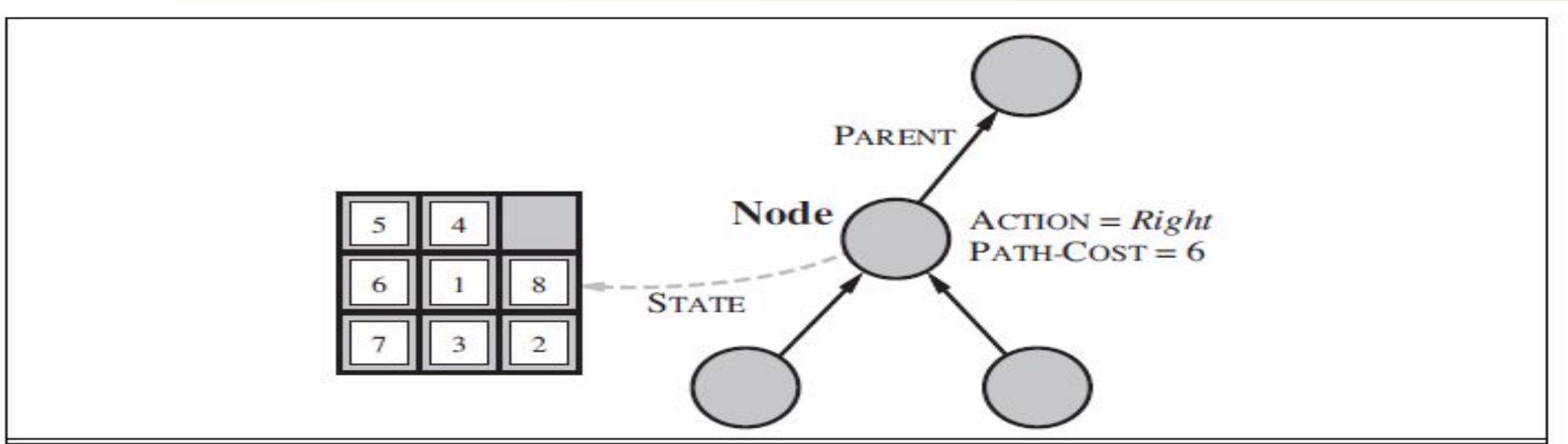


Figure 3.10 Nodes are the data structures from which the search tree is constructed. Each has a parent, a state, and various bookkeeping fields. Arrows point from child to parent.

PATH-COST = 6, which means:

- 6 actions (moves) were taken from the start state to reach the current state
- Step Cost: Each action (e.g., moving a tile in the 8-puzzle) may have a cost. In many problems like the 8-puzzle, this is often 1 per move, but in other problems, it could vary.

```
function CHILD-NODE(problem, parent, action) returns a node
  return a node with
      STATE = problem.RESULT(parent.STATE, action),
      PARENT = parent, ACTION = action,
      PATH-COST = parent.PATH-COST + problem.STEP-COST(parent.STATE, action)
```

- A node is a bookkeeping data structure used to represent the search tree. A state corresponds to a configuration of the world. Thus, nodes are on particular paths, as defined by PARENT pointers, whereas states are not.

- The frontier needs to be stored in such a way that the search algorithm can easily choose the next node to expand according to its preferred strategy. The appropriate data structure for this is a queue.
- The operations on a queue are as follows:
  - EMPTY?(queue) returns true only if there are no more elements in the queue.
  - POP(queue) removes the first element of the queue and returns it.
  - INSERT(element, queue) inserts an element and returns the resulting queue.

- Three common variants are the first-in, first-out FIFO QUEUE or FIFO queue, which pops the oldest element of the queue;
- LIFO QUEUE the last-in, first-out or LIFO queue (also known as a stack), which pops the newest element
- PRIORITY QUEUE of the queue; and the priority queue, which pops the element of the queue with the highest priority according to some ordering function

2.1.2 Measuring problem-solving performance

- Completeness: Is the algorithm guaranteed to find a solution when there is one?
- Optimality: Does the strategy find the optimal solution?
- Time complexity: How long does it take to find a solution?

- Space complexity: How much memory is needed to perform the search?
- In theoretical computer science, the typical measure is the size of the state space graph, |V | + |E|, where V is the set of vertices (nodes) of the graph and E is the set of edges (links).
- In AI, the graph is often represented implicitly by the initial state, actions, and transition model and is frequently infinite.
- For these reasons, complexity is expressed in terms of three quantities: b, the branching factor or maximum number of successors of any node; d, the depth of the shallowest goal node (i.e., the number of steps along the path from the root); and m, the maximum length of any path in the state space.
- To assess the effectiveness of a search algorithm, we can consider just the search cost— which typically depends on the time complexity but can also include a term for memory usage—or we can use the total cost, which combines the search cost and the path cost of the solution found.

2.2 Uninformed search strategies

- The term **Uninformed search/ blind search** means that the strategies have no additional information about states beyond that provided in the problem definition.
- All they can do is generate successors and distinguish a goal state from a non-goal state. All search strategies are distinguished by the order in which nodes are expanded.
- Strategies that know whether one non-goal state is "more promising" than another are called **informed search** or **heuristic search** strategies

2.2.1 Breadth-first search

- Breadth-first search is a simple strategy in which the root node is expanded first, then all the successors of the root node are expanded next, then their successors, and so on. Breadth-first search is an instance of the general graph-search algorithm in which the shallowest unexpanded

Breadth-First Search (BFS) : Finds the shortest (shallowest) path to a goal.

- **Type:** Instance of general graph search.
- **Strategy:** Expands the shallowest unexpanded node first.
- **Frontier:** Uses a FIFO queue (First-In, First-Out).
- **Node Expansion:**
    - ❑ New (deeper) nodes added to the back of the queue.
    - ❑ Older (shallower) nodes are expanded first.
- **Goal Test:** Applied at node generation, not during expansion.
- **Duplicates:**
    - ❑ Ignored if already in the frontier or explored set.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
    node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    frontier ← a FIFO queue with node as the only element
    explored ← an empty set
    loop do
        if EMPTY?(frontier) then return failure
        node ← POP(frontier)  /* chooses the shallowest node in frontier */
        add node.STATE to explored
        for each action in problem.ACTIONS(node.STATE) do
            child ← CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
                frontier ← INSERT(child, frontier)
```

Figure 3.11 Breadth-first search on a graph.

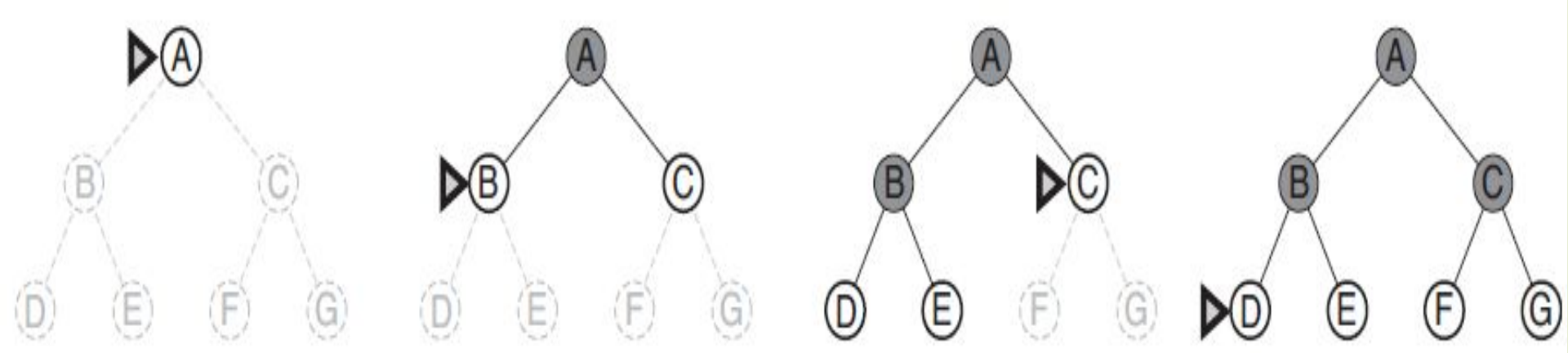


Figure: Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

Breadth-First Search – Time & Space Complexity:

Branching Factor: b(each node has b children)

Depth of Solution: d

Time Complexity:

•Nodes generated: $b + b^2 + b^3 + \cdots + b^d = O(b^d)$ worst case)

•If goal test is applied at expansion, time becomes: $O(b^{d+1})$

Space Complexity:

All nodes are kept in memory (frontier + explored):

•Explored set: $O(b^{d-1})$

•Frontier: $O(b^d)$

•So total space: $O(b^d)$

2.2.2 Uniform-cost search

- Instead of expanding the shallowest node, uniform-cost search expands the node n with the lowest path cost g(n).

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Figure 3.14 Uniform-cost search on a graph. The algorithm is identical to the general graph search algorithm in Figure 3.7, except for the use of a priority queue and the addition of an extra check in case a shorter path to a frontier state is discovered. The data structure for *frontier* needs to support efficient membership testing, so it should combine the capabilities of a priority queue and a hash table.

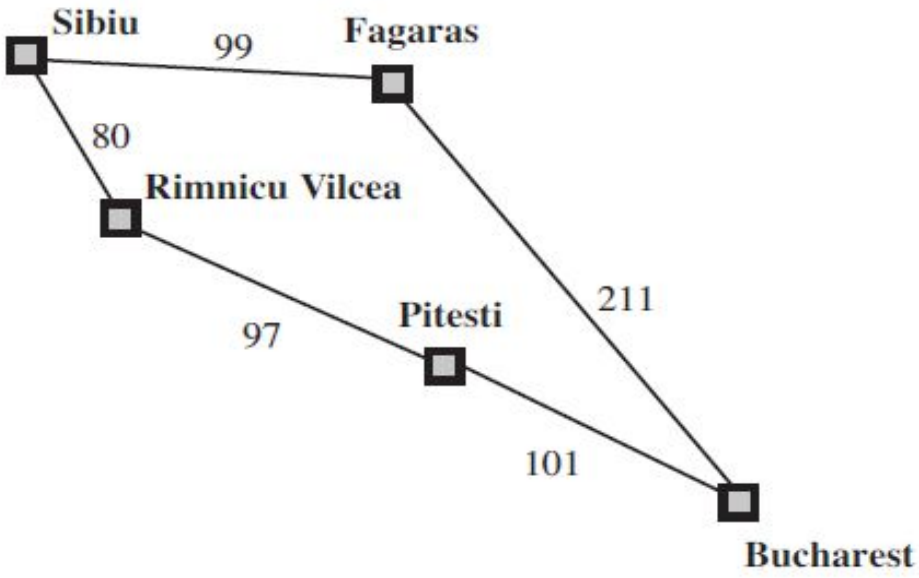


Figure 3.15 Part of the Romania state space, selected to illustrate uniform-cost search.

- The problem is to get from Sibiu to Bucharest. The successors of Sibiu are Rimnicu Vilcea and Fagaras, with costs 80 and 99, respectively.
- The least-cost node, Rimnicu Vilcea, is expanded next, adding Pitesti with cost 80 + 97=177. The least-cost node is now Fagaras, so it is expanded, adding Bucharest with cost 99+211=310.

- Now a goal node has been generated, but uniform-cost search keeps going, choosing Pitesti for expansion and adding a second path to Bucharest with cost 80+97+101= 278. Now the algorithm checks to see if this new path is better than the old one; it is, so the old one is discarded. Bucharest, now with g-cost 278, is selected for expansion and the solution is returned.
- uniform-cost search expands nodes in order of their optimal path cost.

2.2.3 Depth-first search

- Depth-first search always expands the deepest node in the current frontier of the search tree.
- Depth-first search uses a LIFO queue. A LIFO queue means that the most recently generated node is chosen for expansion.
- This must be the deepest unexpanded node because it is one deeper than its parent—which, in turn, was the deepest unexpanded node when it was selected.

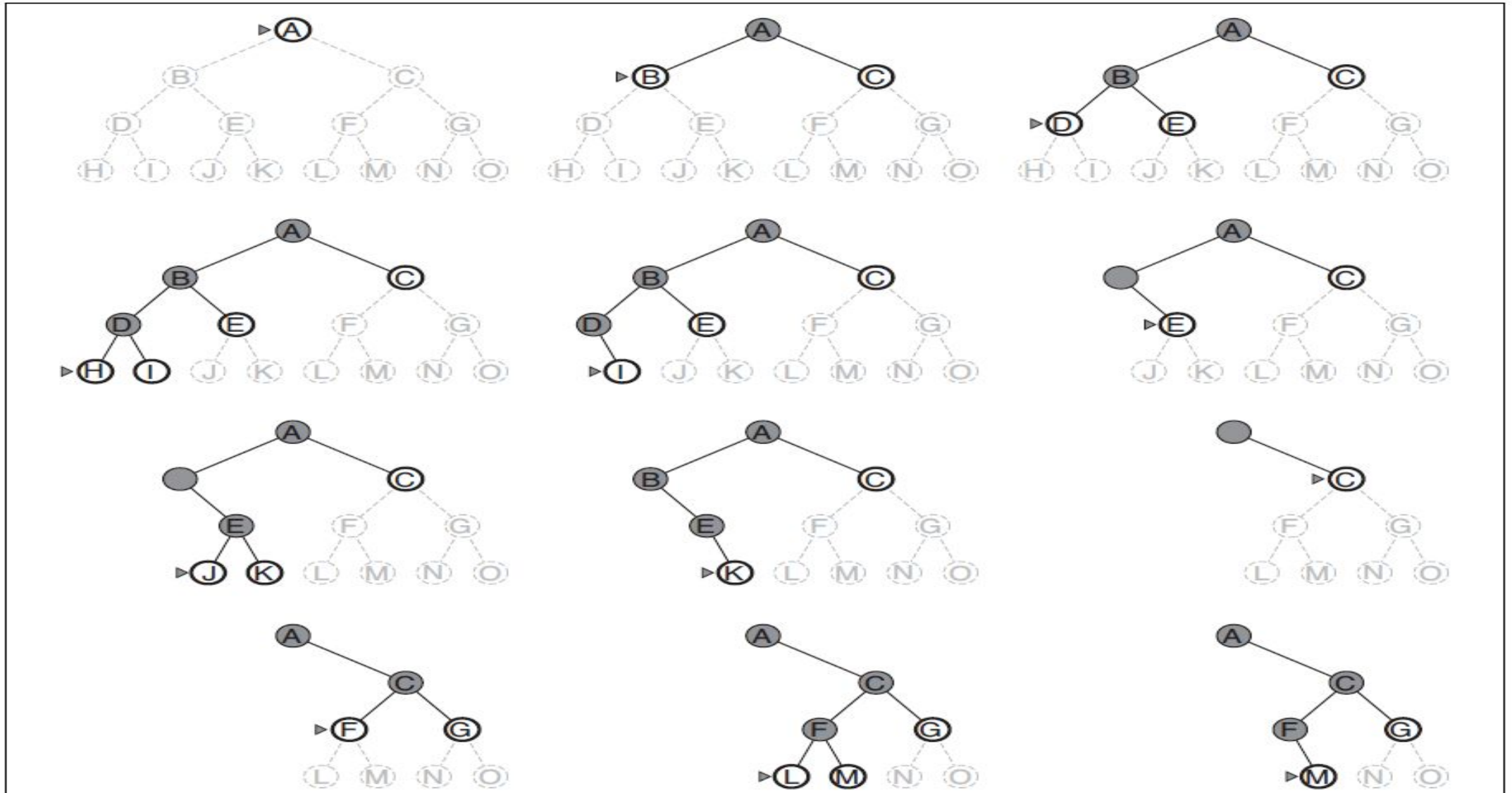


Figure 3.16 Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and M is the only goal node.

Tree vs Graph Search

- **Graph-search DFS:**
  - ❑ Avoids repeated states and redundant paths.
  - ❑ Complete in finite state spaces.
- **Tree-search DFS:**
  - ❑ Not complete (can loop forever, e.g., Arad–Sibiu loop).
  - ❑ Can be improved by checking states on the current path (avoids infinite loops but not redundant paths).
- Depth-first tree search can be modified at no extra memory cost so that it checks new states against those on the path from the root to the current node; this avoids infinite loops in finite state spaces but does not avoid the proliferation of redundant paths. In infinite state spaces,

Time Complexity:

DFS Graph Search: Bounded by state space size (may be infinite).

DFS Tree Search: Can generate up to O(b^m) nodes

- b:branching factor
- m:max depth (can be >>d, or infinite)

Space Complexity:

DFS Tree Search: Stores only the current path and unexpanded siblings → O(bm) space

- Much less than Breadth-First Search (BFS) which requires O(b^d) space
- Example: At depth 16, DFS uses 156 KB vs BFS's 10 exabytes

Backtracking Search (DFS Variant):

- Generates one successor at a time
- Requires only O(m) memory O(bm)
- Ideal for large state problems (e.g., robotics)

2.2.4 Depth-limited search

•The embarrassing failure of depth-first search in infinite state spaces can be alleviated by supplying depth-first search with a predetermined depth limit l. That is, nodes at depth l are treated as if they have no successors. This approach is called depth-limited search.

Unfortunately, it also introduces an additional

- source of incompleteness if we choose $l < d$, that is, the shallowest goal is beyond the depth limit. (This is likely when d is unknown.) Depth-limited search will also be nonoptimal if we choose $l > d$. Its time complexity is $O(b^l)$ and its space complexity is $O(b^l)$. Depth-first search can be viewed as a special case of depth-limited search with $l=\infty$.
- For example, on the map of Romania there are 20 cities. Therefore, we know that if there is a solution, it must be of length 19 at the longest, so $l = 19$ is a possible choice. But in fact if we studied the map carefully, we would discover that any city can be reached from any other city in at most 9 steps.
- This number, known as the **diameter** of the state space, gives us a better depth limit, which leads to a more efficient depth-limited search.
- Depth-limited search can terminate with two kinds of failure: the standard failure value indicates

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

Figure 3.17 A recursive implementation of depth-limited tree search.

2.2.5 Iterative deepening depth-first search

- Iterative deepening search (or iterative deepening depth-first search) is a general strategy, often used in combination with depth-first tree search, that finds the best depth limit. It does this by gradually increasing the limit—first 0, then 1, then 2, and so on—until a goal is found.
- This will occur when the depth limit reaches d, the depth of the shallowest goal node.
- Iterative deepening combines the benefits of depth-first and breadth-first search. Like depth-first search, its memory requirements are modest: O(bd) to be precise. Like breadth-first search, it is complete when the branching factor is finite and optimal when the path cost is a

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    for depth = 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```

Figure 3.18 The iterative deepening search algorithm, which repeatedly applies depth-limited search with increasing limits. It terminates when a solution is found or if the depth-limited search returns *failure*, meaning that no solution exists.

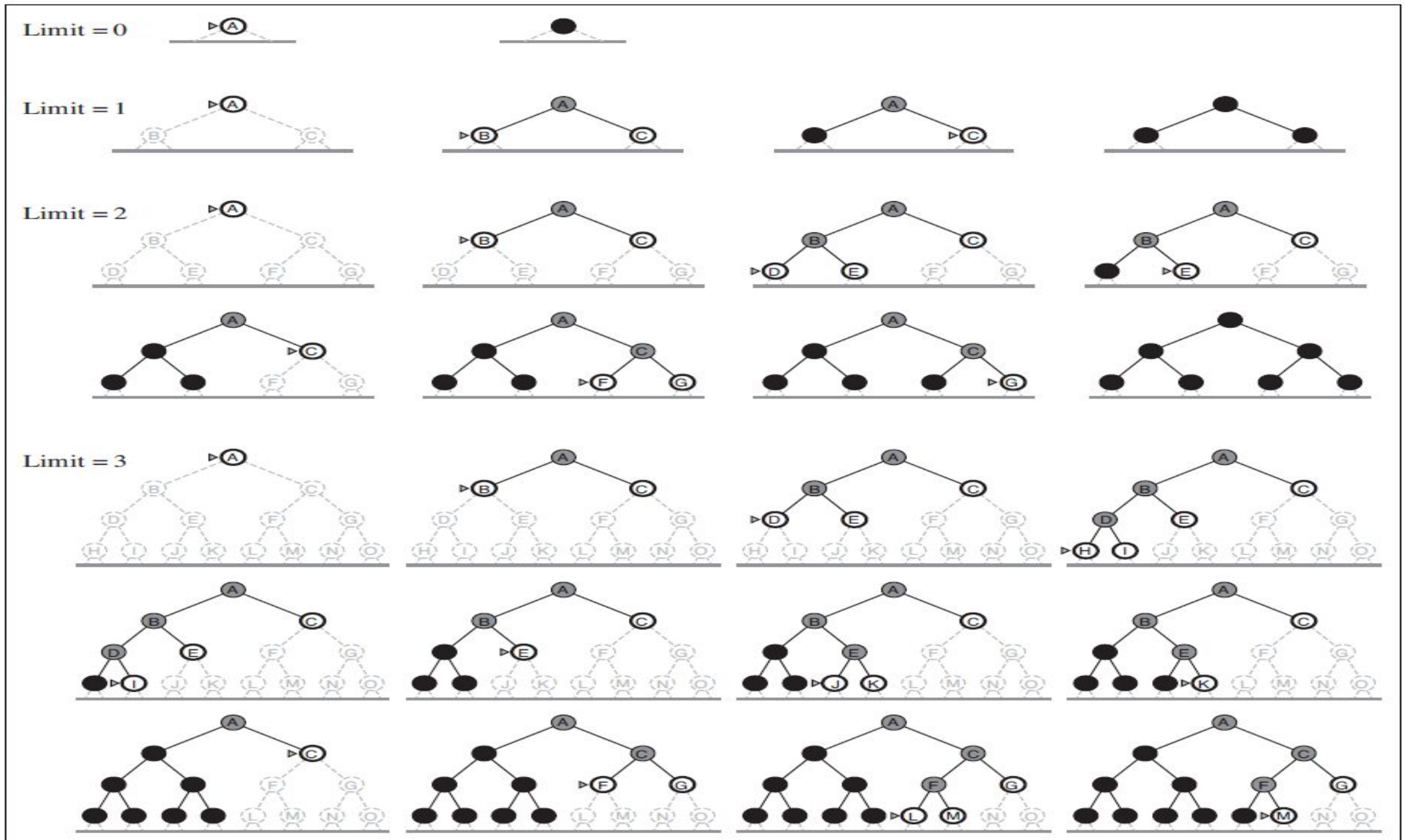


Figure 3.19 Four iterations of iterative deepening search on a binary tree.

- Each row shows a search iteration with a specific depth limit (Limit=0, Limit=1, etc.)
- Each box (subfigure) within the row represents a step in that iteration.
- Black nodes indicate nodes that were generated/expanded (visited).
- Gray nodes are currently in the frontier or about to be expanded.
- White nodes with dotted lines are not yet generated
- The triangular Pointer marks the current node being expanded
- **Limit = 0**
    - ❑ Only the root node A is generated.
    - ❑ No child nodes are explored because the depth limit is 0.
    - ❑ All four steps show how the algorithm terminates quickly at this depth.

Limit = 1

- IDS performs DFS to depth 1.
- Root node A is expanded → children B and C are generated.
- Each of the 4 steps shows different orders of node expansions at depth 1.
- But nodes at depth 2 (like D, E, etc.) are not generated because the depth limit is 1.

Limit = 2

- Depth limit increases to 2.
- The tree is now explored deeper.
- After expanding A → B → D and E, we now reach depth 2.
- Nodes D, E, F, G are explored in the process.
- Multiple traversal sequences are shown in the 4 subfigures.

Limit = 3

- The depth limit is now 3, so the tree can be searched further.
- Nodes like H, I, J, K, L, M (which are children of D, E, F, etc.) are now generated.
- Each subfigure shows a different ordering of the same iterative deepening search process.
- More nodes are visited compared to previous limits, but the search is still depth-first and bounded.

Note:

- In general, iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is not known.
- BFS is best when memory is not a constraint and shortest path is needed.

2.2.6 Bidirectional search

- The idea behind bidirectional search is to run two simultaneous searches—one forward from the initial state and the other backward from the goal—hoping that the two searches meet in the middle.
- The motivation is that b^d/2 + b^d/2 is much less than b^d, or in the figure, the area of the two small circles is less than the area of one big circle centered on the start and reaching to the goal.
- Bidirectional search is implemented by replacing the goal test with a check to see whether the frontiers of the two searches intersect; if they do, a solution has been found.
- The time complexity of bidirectional search using breadth-first searches in both directions is O(b^d/2). The space complexity is also O(b^d/2).
- We can reduce this by roughly half if one of the two searches is done by iterative deepening,

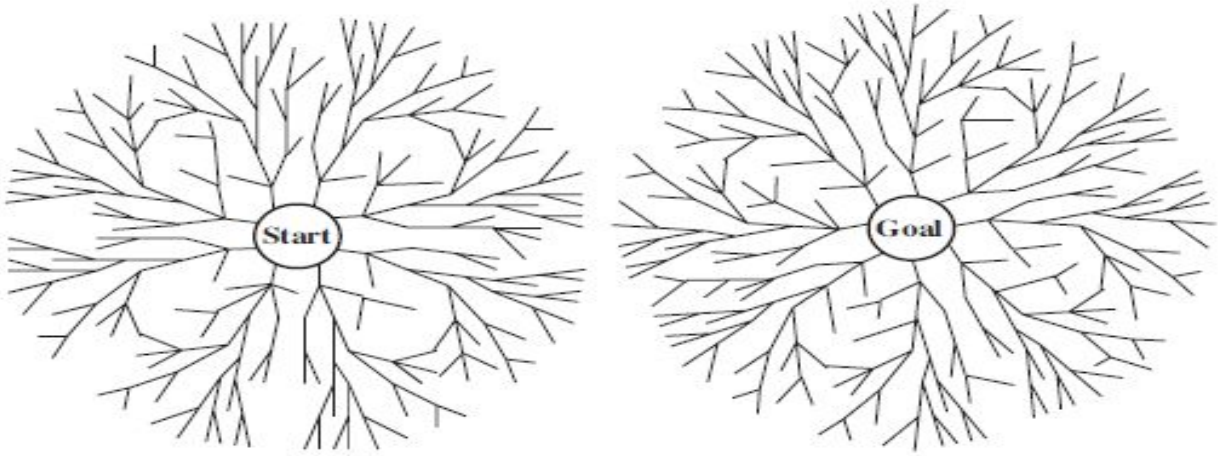


Figure 3.20 A schematic view of a bidirectional search that is about to succeed when a branch from the start node meets a branch from the goal node.

2.2.7 Comparing uninformed search strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search.

2.3 Informed (HEURISTIC) search strategies

- Informed search strategy—one that uses problem-specific knowledge beyond the definition of the problem itself—can find solutions more efficiently than can an uninformed strategy.
- The general approach we consider is called best-first search. Best-first search is an instance of the general TREE-SEARCH or GRAPH-SEARCH algorithm in which a node is selected for expansion based on an evaluation function, f(n).
- The evaluation function is construed as a cost estimate, so the node with the lowest evaluation is expanded first.
- Most best-first algorithms include as a component of f a heuristic function, denoted h(n):

h(n) = estimated cost of the cheapest path from the state at node n to a goal state

2.3.1 Greedy best-first search

- Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly. Thus, it evaluates nodes by using just the heuristic function; that is, f(n) = h(n).

- Let us see how this works for route-finding problems in Romania; we use the straight line distance heuristic, which we will call h_{SLD}. If the goal is Bucharest, we need to know the straight-line d

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Figure 3.22 Values of h_{SLD}—straight-line distances to Bucharest.

2.3.2 A* search: Minimizing the total estimated solution cost

- The most widely known form of best-first search is called **A*** search (pronounced "A-star search"). It evaluates nodes by combining g(n), the cost to reach the node, and h(n), the cost to get from the node to the goal:

$$f(n) = g(n) + h(n)$$

- Since g(n) gives the path cost from the start node to node n, and
- h(n) is the estimated cost of the cheapest path from n to the goal
- f(n) = estimated cost of the cheapest solution through n .
- Reasonable thing to try first is the node with the lowest value of g(n) + h(n). It turns out that this strategy is more than just reasonable: provided that the heuristic function h(n) satisfies certain conditions, A* search is both complete and optimal. The algorithm is identical to

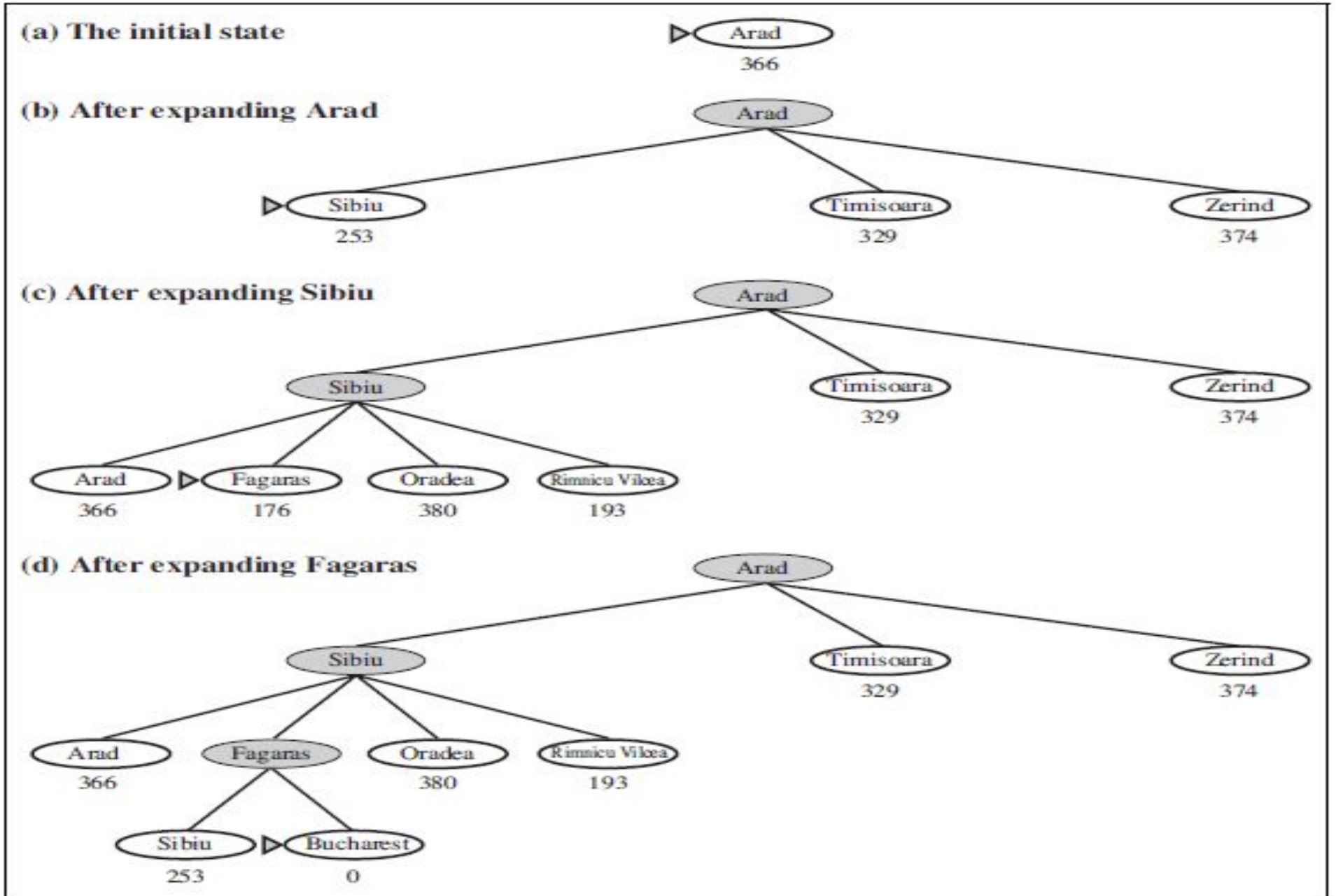
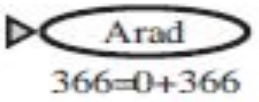


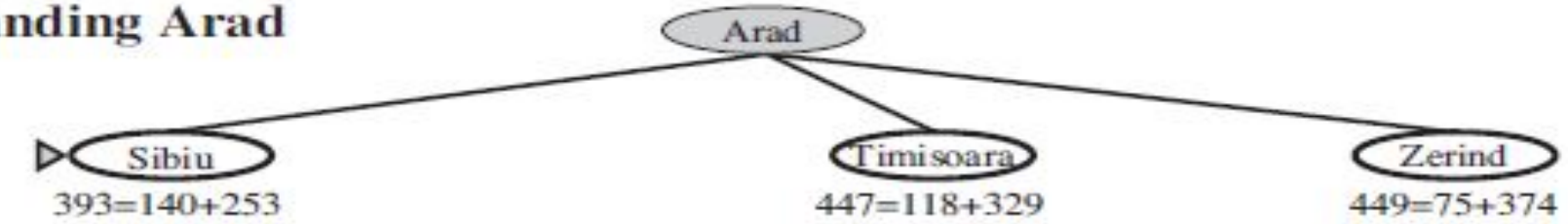
Figure 3.23 Stages in a greedy best-first tree search for Bucharest with the straight-line distance heuristic h_{SLD}. Nodes are labeled with their h-values.

- Conditions for optimality: Admissibility and consistency
- The first condition we require for optimality is that h(n) be an admissible heuristic. An admissible heuristic is one that never overestimates the cost to reach the goal.
- Because g(n) is the actual cost to reach n along the current path, and f(n)=g(n) + h(n), we have as an immediate consequence that f(n) never overestimates the true cost of a solution along the current path through n.

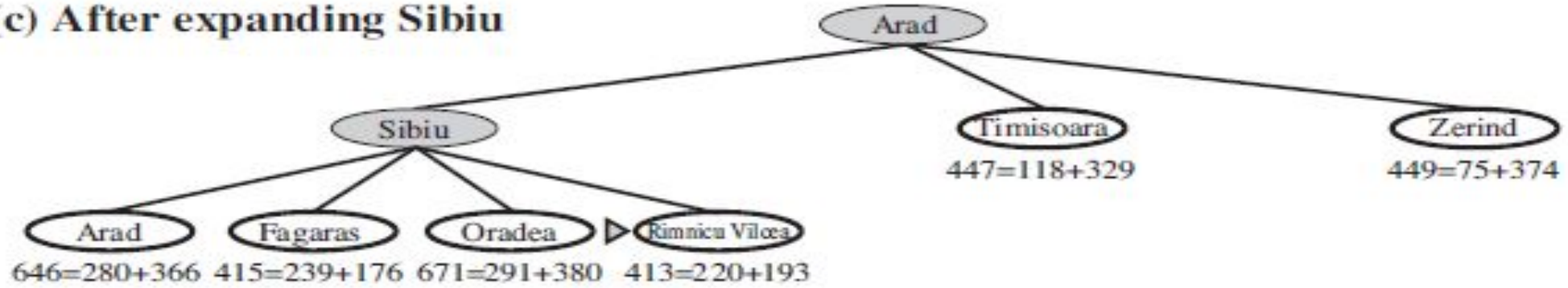
(a) The initial state



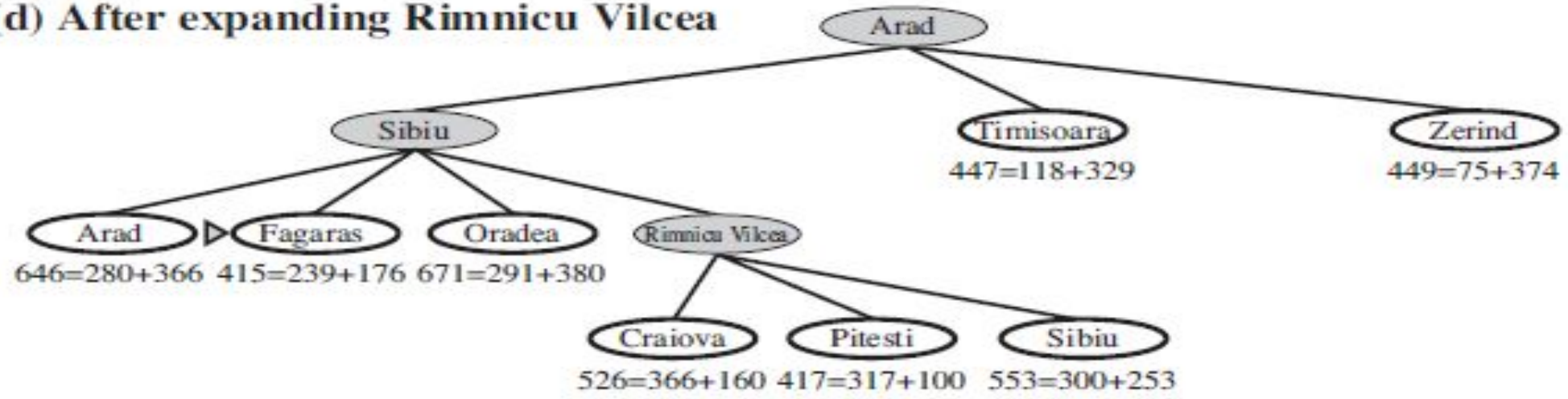
(b) After expanding Arad



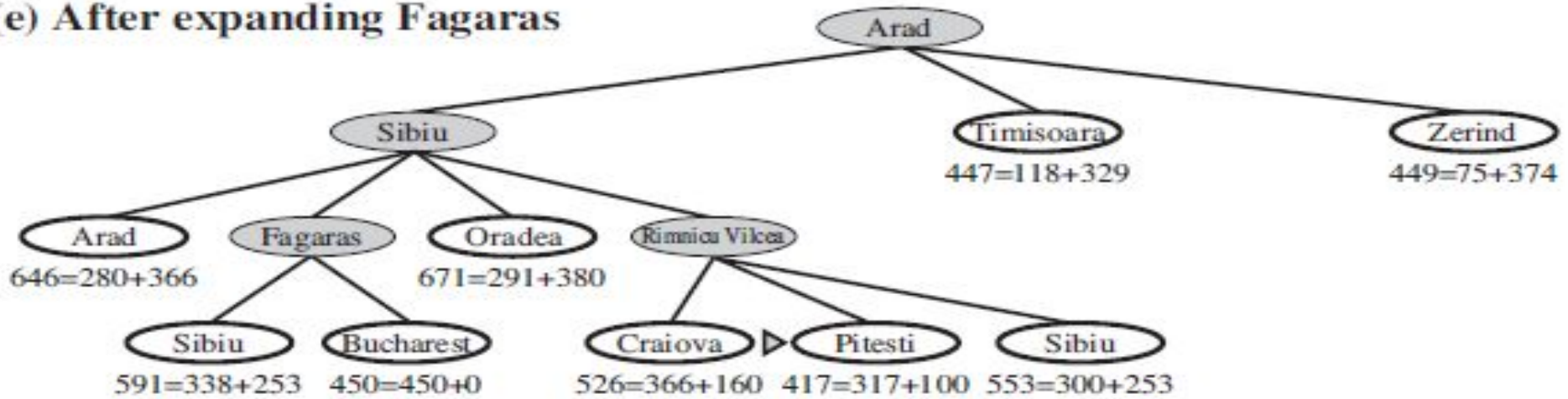
(c) After expanding Sibiu



(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



(f) After expanding Pitesti

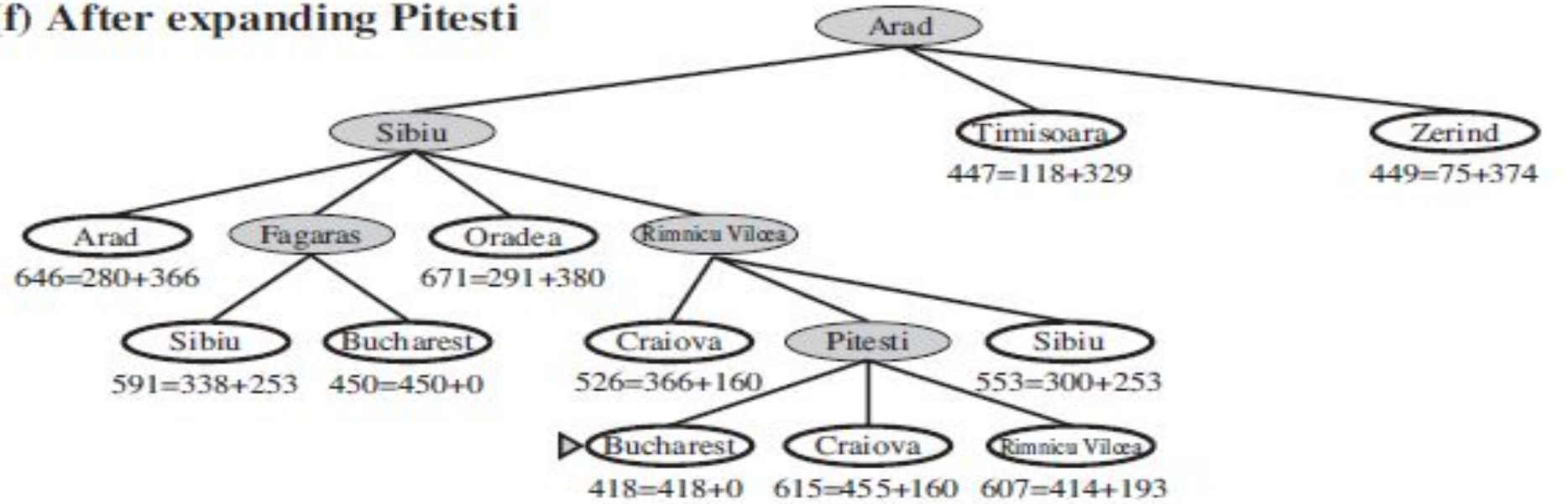


Figure 3.24 Stages in an A* search for Bucharest. Nodes are labeled with $f = g + h$. The h values are the straight-line distances to Bucharest taken from Figure 3.22.

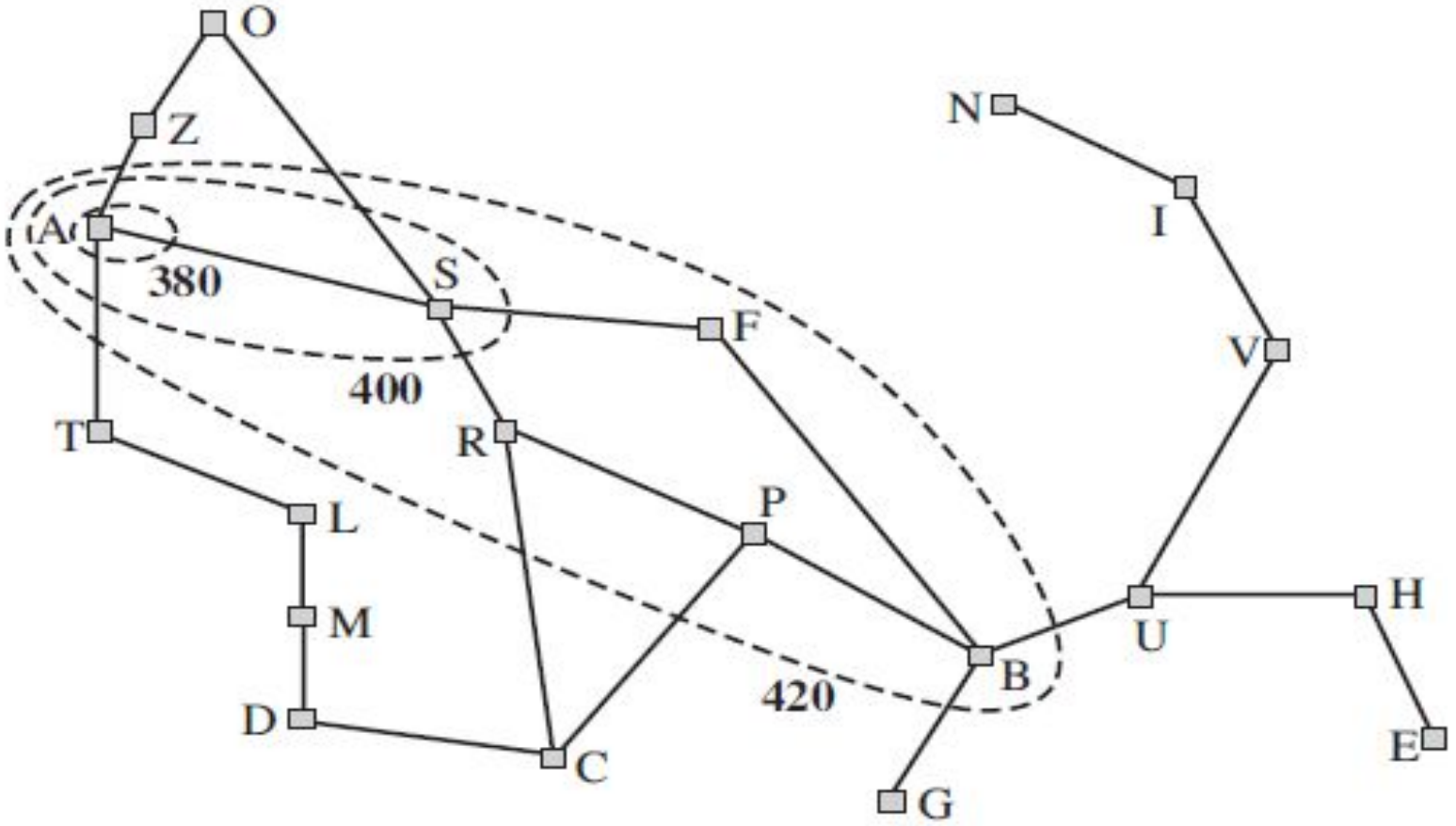


Figure 3.25 Map of Romania showing contours at $f = 380$, $f = 400$, and $f = 420$, with Arad as the start state. Nodes inside a given contour have f-costs less than or equal to the contour value.